

PROGRAMACIÓN APLICADA

MANUAL DE

GIT Y GITHUB

Guía práctica, progresiva y profesional para estudiantes que comienzan desde cero

GITHUB • colaboración, respaldo y revisión

GIT • historial, ramas y control de versiones

TU PROYECTO • código fuente y documentación

Edición 2026-2

Material de apoyo para clases, laboratorios y proyectos semestrales

Cómo utilizar este manual

Este documento está diseñado para estudiantes que pueden saber programar algo, pero todavía no dominan el trabajo profesional con control de versiones. Puede estudiarse de principio a fin o utilizarse como referencia rápida durante el desarrollo de un proyecto.

Objetivo principal — Al finalizar, el estudiante debe ser capaz de crear y configurar repositorios, registrar cambios con criterio, trabajar con ramas, sincronizarse con GitHub, colaborar mediante Pull Requests, resolver conflictos y recuperar errores comunes sin perder el control del proyecto.

Resultados de aprendizaje

- Explicar con precisión la diferencia entre Git y GitHub.
- Comprender repositorio, commit, rama, HEAD, staging area, remoto, merge y Pull Request.
- Crear un repositorio local y publicarlo correctamente en GitHub.
- Utilizar add, commit, status, log, diff, restore, reset, fetch, pull, push, branch, switch y merge.
- Trabajar en equipo evitando que todos programen directamente sobre main.
- Resolver conflictos de merge de forma razonada.
- Recuperarse de errores comunes sin recurrir a “borrar todo y empezar de nuevo”.
- Aplicar un flujo de trabajo adecuado a proyectos académicos de Backend, Web y Android.

Convenciones utilizadas

Elemento	Significado
Comando	Instrucción que se ejecuta en la terminal.
Repositorio local	Copia Git que se encuentra en tu computadora.
Repositorio remoto	Repositorio alojado en GitHub u otro servidor.
main	Rama principal del proyecto.
feature/...	Rama temporal destinada a una funcionalidad concreta.
ORIGEN	Nombre lógico del remoto; normalmente origin.

Regla del manual — Nunca copies un comando sin saber en qué carpeta estás, qué archivos cambiará y si afecta únicamente a tu equipo o también al repositorio remoto.

Índice de contenidos

1. Qué problema resuelve el control de versiones
2. Qué es Git
3. Qué es GitHub
4. Modelo mental de Git
5. Instalación y configuración inicial
6. Primer repositorio desde cero
7. El ciclo diario de trabajo
8. GitHub: publicar y clonar repositorios
9. Ramas: trabajar sin romper main
10. Merge y resolución de conflictos
11. Pull Requests y revisión de código
12. Trabajo profesional en equipo
13. Deshacer cambios y recuperarse de errores
14. .gitignore y archivos que no deben versionarse
15. Tags, releases y versiones
16. Estrategia para proyectos de Programación Aplicada
17. Flujo con IntelliJ IDEA, WebStorm, Android Studio y DataGrip
18. Seguridad: credenciales, secretos y buenas prácticas
19. Diagnóstico de errores frecuentes
20. Laboratorios guiados
21. Evaluación práctica sugerida
22. Hoja de referencia rápida
23. Glosario esencial
24. Checklist antes de entregar un proyecto
25. Diez reglas para no sufrir con Git

1. Qué problema resuelve el control de versiones

Cuando un proyecto crece, aparecen varias preguntas: ¿qué cambió?, ¿quién lo cambió?, ¿por qué se cambió?, ¿qué versión funcionaba ayer?, ¿cómo integramos el trabajo de varios integrantes?, ¿cómo experimentamos sin destruir lo que ya funciona? Un sistema de control de versiones responde precisamente a estas preguntas.

1.1 El método incorrecto: copias de carpetas

Lo que ocurre cuando no se usa control de versiones

```
proyecto_final/  
proyecto_final_bueno/  
proyecto_final_bueno_2/  
proyecto_final_ahora_si/  
proyecto_final_ahora_si_FINAL/  
proyecto_final_ahora_si_FINAL_v2/
```

Las copias manuales no describen con precisión qué cambió, generan archivos duplicados, hacen muy difícil fusionar trabajo de varias personas y no crean un historial confiable. Git sustituye este caos por una línea de tiempo estructurada de cambios.

1.2 Qué ofrece un VCS moderno

- Historial: cada cambio relevante queda registrado.
- Trazabilidad: es posible saber quién creó un cambio y con qué mensaje.
- Comparación: se pueden ver diferencias línea por línea.
- Recuperación: se puede volver a estados anteriores.
- Paralelismo: varias personas pueden trabajar en ramas distintas.
- Integración: los cambios se fusionan de forma controlada.
- Auditoría técnica: el historial ayuda a analizar la evolución real del proyecto.

2. Qué es Git

Git es un sistema distribuido de control de versiones. “Distribuido” significa que cada desarrollador que clona un repositorio obtiene una copia del historial del proyecto, no solamente los archivos actuales. Por eso Git puede realizar muchas operaciones localmente, incluso sin conexión a Internet.

2.1 Git no es un almacenamiento de archivos cualquiera

Git está optimizado para registrar estados coherentes del proyecto. Cada commit representa una fotografía lógica del contenido versionado y mantiene referencias que permiten reconstruir la historia. Aunque internamente Git utiliza objetos y hashes, para comenzar basta con dominar la idea de estados, commits y referencias.

2.2 Git trabaja localmente

Idea fundamental — Puedes usar Git sin GitHub. `git init`, `git add`, `git commit`, `git log`, `git branch` y muchas otras operaciones funcionan completamente en tu computadora. GitHub aparece cuando quieres alojar, compartir y colaborar sobre ese repositorio.

3. Qué es GitHub

GitHub es una plataforma de alojamiento y colaboración basada en Git. Permite almacenar repositorios remotos y agregar capacidades de trabajo en equipo: Pull Requests, revisión de código, Issues, Releases, automatizaciones, controles de acceso y más.

3.1 Git y GitHub no son lo mismo

Git	GitHub
Software de control de versiones	Plataforma de alojamiento y colaboración
Funciona en tu computadora	Funciona principalmente como servicio remoto
Registra commits y ramas	Aloja repositorios y facilita revisión
No necesita Internet para muchas operaciones	Se utiliza por Internet
Comandos: git ...	Interfaz web + integración con Git

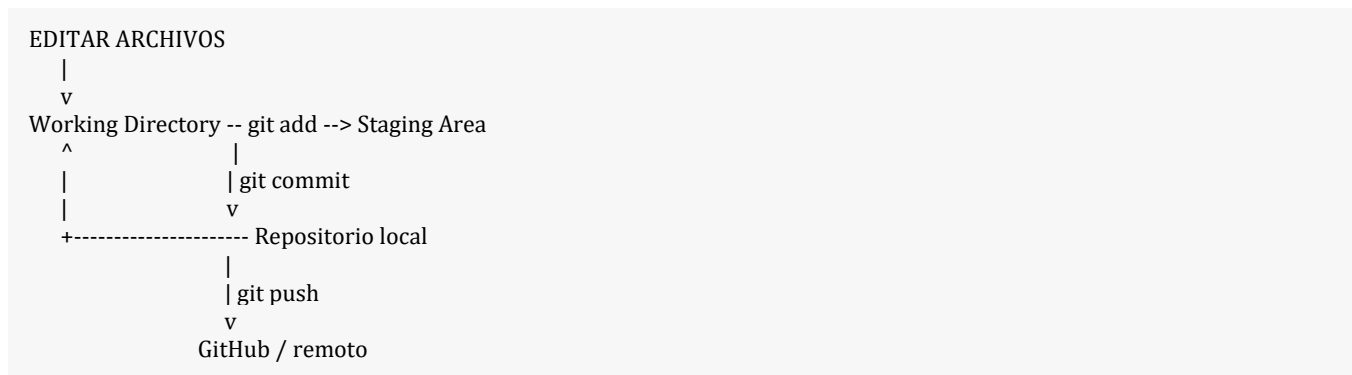
Analogía útil — Git es el motor de control de versiones. GitHub es una plataforma de colaboración que utiliza ese motor. Saber “subir archivos a GitHub” no equivale a saber Git.

4. Modelo mental de Git

Antes de memorizar comandos, hay que entender los espacios por los que pasa un cambio. La mayoría de los errores de principiantes ocurren porque se ejecutan comandos sin saber en qué estado se encuentra cada archivo.

4.1 Los cuatro espacios esenciales

Espacio	Qué representa	Comando relacionado
Working directory	Archivos que estás editando ahora.	git status / git diff
Staging area	Cambios seleccionados para el próximo commit.	git add
Repositorio local	Commits guardados en tu equipo.	git commit / git log
Repositorio remoto	Commits compartidos en GitHub.	git push / git pull



4.2 Qué es un commit

Un commit es un registro identificado de un conjunto de cambios. Incluye autor, fecha, mensaje y una referencia criptográfica. Un buen commit debe representar una unidad lógica: por ejemplo, “crear endpoint de registro de usuarios”, no una mezcla de veinte tareas inconexas.

4.3 Qué es HEAD

HEAD es una referencia que indica en qué punto de la historia estás trabajando. Normalmente apunta a la rama actual, y esa rama apunta a su commit más reciente. Comprender HEAD se vuelve importante cuando se deshacen cambios, se inspecciona el historial o se cambia de rama.

5. Instalación y configuración inicial

Git debe estar instalado en cada computadora que participe en el desarrollo. Después de instalarlo, la primera tarea es configurar la identidad que se registrará en los commits.

5.1 Verificar la instalación

```
git --version
```

Si el comando devuelve una versión, Git está disponible. Si el sistema indica que el comando no existe, debe corregirse la instalación o el PATH.

5.2 Configurar nombre y correo

```
git config --global user.name "Tu Nombre Completo"  
git config --global user.email "tu-correo@ejemplo.com"
```

La opción `--global` aplica esta identidad a los repositorios del usuario actual. En proyectos donde se necesita otra identidad, puede configurarse localmente sin `--global` dentro del repositorio.

5.3 Revisar configuración

```
git config --global --list
```

5.4 Configurar main como nombre por defecto

```
git config --global init.defaultBranch main
```

5.5 Saltos de línea: concepto importante

Windows y sistemas Unix representan los saltos de línea de forma distinta. Git puede normalizarlos. Para equipos mixtos, lo más importante es evitar cambios masivos de archivos producidos únicamente por diferencias de final de línea y utilizar configuraciones consistentes.

Consejo para laboratorio — Todos los integrantes del grupo deben verificar su identidad Git antes de realizar el primer commit. De lo contrario, el historial puede atribuir cambios a nombres o correos incorrectos.

6. Primer repositorio desde cero

6.1 Crear una carpeta de proyecto

```
mkdir demo-git  
cd demo-git
```

6.2 Inicializar Git

```
git init
```

git init crea la estructura interna .git. Esa carpeta contiene metadatos, objetos y referencias. No debe editarse manualmente en el trabajo normal.

6.3 Crear contenido y observar el estado

```
echo "# Demo Git" > README.md  
git status
```

El archivo aparece como untracked porque existe en el directorio de trabajo pero todavía no forma parte del historial.

6.4 Llevar el archivo al staging area

```
git add README.md  
git status
```

6.5 Crear el primer commit

```
git commit -m "docs: crear README inicial"
```

6.6 Ver el historial

```
git log --oneline --decorate --graph
```

Regla de oro — Un commit exitoso no significa que el cambio esté en GitHub. El commit vive primero en tu repositorio local. Para compartirlo necesitas un remoto y posteriormente git push.

7. El ciclo diario de trabajo

El flujo diario de Git es repetitivo. Un desarrollador profesional ejecuta status y diff con mucha frecuencia porque estos comandos son de inspección y reducen errores.

7.1 Ciclo mínimo

```
git status
git diff
git add <archivos>
git diff --staged
git commit -m "mensaje claro"
git log --oneline -5
```

7.2 git status

Muestra la rama actual y clasifica los archivos según estén modificados, staged o no rastreados. Debe ser el primer comando ante cualquier duda.

7.3 git diff

Muestra cambios no preparados. git diff --staged muestra lo que efectivamente entrará en el próximo commit. Esta segunda variante es una revisión de calidad antes de registrar la historia.

7.4 git add

```
git add src/main/java/com/ejemplo/UserController.java
git add .
```

git add . agrega todos los cambios visibles bajo la carpeta actual. Es cómodo, pero un principiante debe acostumbrarse a revisar git status antes para no incluir archivos accidentales.

7.5 Mensajes de commit

Malo	Mejor
cambios	feat: agregar registro de estudiantes
arreglo	fix: validar correo duplicado en usuarios
cosas	refactor: extraer mapper de Student
final	docs: actualizar instrucciones de ejecución

Una convención sencilla es utilizar prefijos como feat, fix, docs, refactor, test, chore. Lo importante es que el mensaje explique la intención del cambio.

8. GitHub: publicar y clonar repositorios

8.1 Concepto de remote

Un remote es un alias local que apunta a la ubicación de otro repositorio. Por convención, el principal suele llamarse origin.

```
git remote -v
```

8.2 Vincular un repositorio local con GitHub

```
git remote add origin URL_DEL_REPOSITORIO
git push -u origin main
```

La opción -u establece una relación de seguimiento entre la rama local main y origin/main. Después, normalmente bastará con git push y git pull.

8.3 Clonar un proyecto existente

```
git clone URL_DEL_REPOSITORIO
cd nombre-del-repositorio
git status
```

git clone descarga el repositorio y configura origin automáticamente. Para incorporarse a un proyecto existente, normalmente se clona; no se crea una carpeta aparte con git init.

8.4 fetch, pull y push

Comando	Efecto principal	Idea
git fetch	Descarga información del remoto sin fusionarla automáticamente.	Traer conocimiento
git pull	Descarga e integra cambios en la rama actual.	Traer + integrar
git push	Envía commits locales al remoto.	Compartir

No confundir — push no “sube tu carpeta”; envía objetos/commits que Git necesita transferir. pull no debe ejecutarse mecánicamente si tienes cambios locales desordenados: primero revisa status.

9. Ramas: trabajar sin romper main

Una rama es un nombre móvil que apunta a un commit. Permite desarrollar una funcionalidad de forma aislada. En equipos, las ramas reducen el riesgo de que cada integrante escriba directamente sobre la línea principal.

9.1 Visualización conceptual

```
A---B---C main
      |
      D---E feature/login
```

9.2 Crear y cambiar a una rama

```
git switch -c feature/login
```

Este comando crea la rama y cambia el working directory para trabajar sobre ella. La alternativa clásica es `git checkout -b feature/login`, pero `switch` expresa mejor la intención.

9.3 Listar ramas

```
git branch
git branch -a
```

9.4 Publicar una rama

```
git push -u origin feature/login
```

9.5 Convención sugerida

Prefijo	Uso	Ejemplo
feature/	Nueva funcionalidad	feature/reservas
fix/	Corrección de defecto	fix/login-null
refactor/	Reestructuración sin nueva función	refactor/user-service
docs/	Documentación	docs/api-readme
test/	Pruebas	test/student-service

Disciplina — Cada rama debe tener una intención clara y una vida corta. Una rama que permanece abierta durante semanas acumula diferencias y eleva la probabilidad de conflictos.

10. Merge y resolución de conflictos

10.1 Merge normal

Cuando una funcionalidad está lista, sus commits deben integrarse a la rama destino. Desde Git puede hacerse un merge directo; en equipos con GitHub suele preferirse un Pull Request para revisar antes de integrar.

```
git switch main
git pull
git merge feature/login
```

10.2 Qué es un conflicto

Git puede fusionar automáticamente la mayoría de cambios. Un conflicto aparece cuando no puede decidir cómo combinar modificaciones incompatibles, por ejemplo cuando dos ramas modificaron las mismas líneas de un archivo.

10.3 Marcadores de conflicto

```
<<<<<< HEAD
return "Bienvenido";
=====
return "Bienvenido al sistema";
>>>>>> feature/login
```

Estos marcadores no deben quedar en el código final. El desarrollador debe analizar ambas intenciones, editar el archivo para dejar una solución correcta, probarla y luego marcar el conflicto como resuelto.

10.4 Proceso seguro

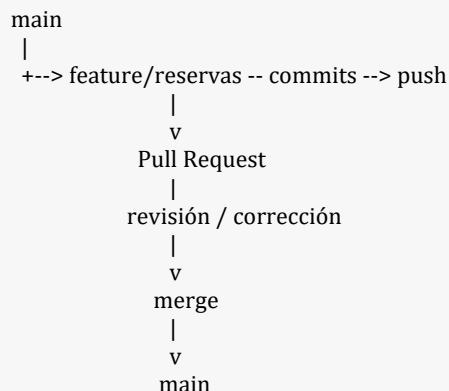
1. Ejecutar `git status` y localizar archivos en conflicto.
2. Abrir cada archivo y comprender ambas versiones.
3. Editar el contenido; no elegir “la mía” o “la de ellos” sin analizar.
4. Compilar y ejecutar pruebas.
5. `git add` sobre los archivos resueltos.
6. Completar el merge mediante `commit` si Git lo requiere.

Nunca haga esto — Borrar el repositorio completo ante un conflicto evita aprender y puede ocultar pérdida de cambios. El conflicto es una situación normal del trabajo colaborativo y debe resolverse conscientemente.

11. Pull Requests y revisión de código

Un Pull Request (PR) es una propuesta para integrar cambios de una rama en otra. No es un comando básico de Git: es un flujo de colaboración proporcionado por plataformas como GitHub.

11.1 Flujo recomendado



11.2 Qué debe contener un PR académico

- Título concreto.
- Descripción de qué se implementó.
- Qué requisito o historia resuelve.
- Cómo probar el cambio.
- Capturas o evidencias cuando aporten valor.
- Advertencias sobre migraciones, variables de entorno o cambios de contrato.
- Checklist de pruebas realizadas.

11.3 Revisión de código

Revisar no significa buscar culpables. Significa proteger la calidad: lógica, legibilidad, seguridad, contratos, manejo de errores, pruebas, consistencia y compatibilidad con el resto del sistema.

Regla de equipo — Nadie debería aprobar un Pull Request que no comprende. La revisión es una responsabilidad técnica, no un clic administrativo.

12. Trabajo profesional en equipo

12.1 Flujo recomendado para grupos

1. Actualizar main antes de comenzar.
2. Crear una rama desde main actualizado.
3. Implementar una unidad de trabajo pequeña.
4. Revisar status y diff.
5. Crear commits lógicos.
6. Subir la rama.
7. Crear Pull Request.
8. Solicitar revisión de al menos un compañero.
9. Corregir observaciones.
10. Integrar a main.

11. Eliminar la rama ya fusionada cuando corresponda.

12.2 Ejemplo de sesión

```
git switch main
git pull
git switch -c feature/student-crud
# ...programar...
git status
git diff
git add src/
git diff --staged
git commit -m "feat: implementar CRUD de estudiantes"
git push -u origin feature/student-crud
```

12.3 Qué NO debe hacer un equipo

- Compartir el proyecto por WhatsApp, USB o ZIP como mecanismo principal de sincronización.
- Usar una sola cuenta de GitHub para todos los integrantes.
- Trabajar todos directamente sobre main.
- Crear commits gigantes al final de la semana.
- Copiar archivos manualmente para “resolver” merges.
- Subir contraseñas, tokens o archivos .env con secretos.
- Forzar push sin comprender sus consecuencias.

13. Deshacer cambios y recuperarse de errores

Git es poderoso porque permite volver atrás, pero los comandos de recuperación no son equivalentes. Antes de ejecutarlos hay que determinar si el cambio está solo en el working directory, ya está staged, ya fue commiteado o incluso ya fue compartido.

13.1 Descartar un cambio local de un archivo

```
git restore ruta/al/archivo
```

Atención — Esta operación elimina modificaciones no guardadas en commits para ese archivo. Si necesitas conservarlas, no la ejecutes hasta tener una copia o un commit.

13.2 Sacar un archivo del staging area

```
git restore --staged ruta/al/archivo
```

El contenido modificado permanece en el working directory; simplemente deja de estar preparado para el próximo commit.

13.3 Corregir el último commit local

```
git commit --amend
```

Amend reemplaza el commit anterior por otro. Es adecuado cuando todavía no se ha compartido. Reescribir commits ya publicados exige más cuidado porque cambia el historial.

13.4 Revertir un commit compartido

```
git revert HASH_DEL_COMMIT
```

revert crea un nuevo commit que invierte el efecto de otro commit. Es especialmente útil en ramas compartidas porque conserva el historial en vez de reescribirlo.

13.5 reset: comprender antes de usar

Variante	Idea general	Riesgo
git reset --soft	Mueve la referencia; conserva cambios preparados.	Moderado
git reset --mixed	Mueve la referencia; conserva archivos modificados.	Moderado
git reset --hard	Mueve la referencia y descarta cambios del working tree.	ALTO

Comando peligroso — git reset --hard puede destruir cambios no confirmados. No debe convertirse en un

“botón para arreglar Git”.

13.6 stash: guardar trabajo temporalmente

```
git stash
git stash list
git stash pop
```

stash guarda temporalmente cambios para limpiar el working directory. Es útil cuando necesitas cambiar de contexto, pero no sustituye una estrategia adecuada de commits.

14. .gitignore y archivos que no deben versionarse

.gitignore define patrones de archivos que Git debe ignorar cuando todavía no están versionados. Es indispensable para evitar basura de compilación, archivos del IDE, dependencias descargables y secretos locales.

14.1 Ejemplo para un ecosistema Java + React

```
# Java / Gradle / Maven
target/
build/
.gradle/

# Node / React
node_modules/
dist/
.next/

# IDEs
.idea/
*.iml

# Entorno y secretos
.env
.env.*

# Sistema operativo
.DS_Store
Thumbs.db
```

14.2 Archivo ya rastreado

Agregar un patrón a .gitignore no elimina mágicamente un archivo que Git ya está siguiendo. Debe dejar de rastrearse explícitamente, cuidando de no perder el archivo local.

```
git rm --cached archivo
git commit -m "chore: dejar de versionar archivo local"
```

Crítico — Un secreto que ya fue commiteado debe considerarse comprometido aunque luego se borre. La acción correcta incluye revocar/rotar la credencial, no solo eliminar el archivo.

15. Tags, releases y versiones

Un tag identifica un punto específico del historial. Es útil para marcar entregas académicas, versiones del MVP o releases estables.

```
git tag -a v0.1.0 -m "Primer MVP demostrable"
git push origin v0.1.0
```

Para proyectos académicos puede ser conveniente etiquetar hitos importantes: v0.1.0-parcial1, v0.2.0-parcial2 y v1.0.0-final, siempre que la convención sea acordada por el docente.

16. Estrategia para proyectos de Programación Aplicada

En un proyecto con Backend, Web y Android, Git debe ser una disciplina transversal. El repositorio no es un simple requisito de entrega; es evidencia de cómo se construyó el software.

16.1 Estructura sugerida de monorepositorio

```
proyecto-grupo/
├── backend/
├── web/
├── mobile/
├── docs/
├── database/
│   ├── migrations/
│   └── seed/
├── .gitignore
├── README.md
└── docker-compose.yml # si corresponde
```

16.2 Alternativa multirrepositorio

También pueden separarse backend, web y mobile en repositorios distintos. Para estudiantes novatos, un monorepositorio suele reducir fricción operativa siempre que la estructura y responsabilidades estén claras. En proyectos más grandes, múltiples repositorios pueden tener sentido.

16.3 Política mínima recomendada

- ☐ main debe mantenerse ejecutable.
- ☐ Cada funcionalidad relevante se desarrolla en una rama.
- ☐ Los commits deben ser frecuentes y significativos.
- ☐ Los Pull Requests deben ser revisados.
- ☐ Los integrantes deben usar cuentas individuales.
- ☐ Las entregas deben identificarse con tags o releases.
- ☐ README debe explicar cómo levantar cada componente.
- ☐ Las migraciones de base de datos deben versionarse; no se comparte una base “mágica” sin scripts.

16.4 Ejemplo de ramas por tarea

```
feature/backend-student-api  
feature/web-student-list  
feature/mobile-student-login  
fix/backend-duplicate-email  
docs/installation-guide
```

16.5 Evidencia de participación individual

El historial de Git puede ayudar a observar contribuciones, pero no debe interpretarse mecánicamente por número de commits. Un estudiante puede crear muchos commits triviales y no comprender el proyecto. En una evaluación seria, el historial se combina con defensa individual, revisión del código y explicación técnica.

17. Flujo con IntelliJ IDEA, WebStorm, Android Studio y DataGrip

Los IDE modernos integran Git, pero la interfaz gráfica no elimina la necesidad de entender los conceptos. La recomendación docente es aprender primero el flujo en terminal y después utilizar las herramientas visuales como aceleradores.

17.1 IntelliJ IDEA / WebStorm / Android Studio

- Muestran archivos modificados y diferencias.
- Permiten preparar cambios para commit.
- Permiten crear/cambiar ramas.
- Facilitan merge y resolución visual de conflictos.
- Pueden hacer push, pull y fetch.
- Integran historial y anotación de líneas.

Criterio — Si una ventana del IDE dice “Commit”, “Push”, “Update”, “Merge” o “Rebase”, el estudiante debe poder explicar cuál es el equivalente conceptual en Git.

17.2 DataGrip y la base de datos

DataGrip administra conexiones y consultas, pero Git debe versionar scripts y migraciones, no la instancia viva de PostgreSQL. Una base de datos local no se “sube” como si fuera código fuente. El proyecto debe poder recrear su estructura mediante migraciones o scripts versionados.

17.3 Postman

Las colecciones y archivos de entorno sin secretos pueden versionarse cuando aporten reproducibilidad. Tokens reales, contraseñas y claves privadas no deben incluirse en Git.

18. Seguridad: credenciales, secretos y buenas prácticas

18.1 Nunca versionar

- Contraseñas reales.
- Tokens personales de acceso.
- Llaves privadas SSH.
- Archivos .env con secretos.
- Credenciales de bases de datos compartidas.
- API keys de servicios externos.
- Certificados privados.

18.2 Configuración mediante variables de entorno

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=app
DB_USER=app_user
DB_PASSWORD=NO_VERSIONAR_ESTO_VALOR
```

El repositorio puede incluir un `.env.example` con nombres de variables y valores ficticios para documentar la configuración, pero el archivo real debe permanecer fuera del control de versiones.

18.3 Autenticación con GitHub

Dependiendo de la configuración, GitHub puede utilizar credenciales mediante un gestor, token o SSH. El principio es el mismo: las credenciales son personales, no se comparten entre integrantes ni se guardan dentro del repositorio.

19. Diagnóstico de errores frecuentes

Síntoma	Qué revisar primero	Acción típica
nothing to commit	¿Realmente cambió un archivo?	git status / guardar archivo
rejected push	El remoto tiene commits nuevos.	fetch/pull y luego integrar
merge conflict	Dos historias tocaron contenido incompatible.	resolver archivos y probar
detached HEAD	HEAD apunta a un commit, no a rama.	crear/cambiar a una rama adecuada
untracked files	Archivos nuevos no añadidos.	git add o .gitignore
secret pushed	Credencial expuesta.	revocar/rotar y limpiar historial si aplica
wrong branch	Trabajo hecho en rama equivocada.	analizar commits antes de moverlos

19.1 Secuencia universal de diagnóstico

```
git status
git branch --show-current
git log --oneline --decorate --graph -10
git remote -v
```

Estos cuatro comandos responden gran parte de las preguntas iniciales: qué está modificado, en qué rama estoy, cuál es mi historia reciente y a qué remoto estoy conectado.

20. Laboratorios guiados

Laboratorio 1 — Mi primer historial

Objetivo: comprender working directory, staging area y commit.

- ☐ Crear carpeta laboratorio-git.
- ☐ Ejecutar git init.
- ☐ Crear README.md.
- ☐ Ejecutar git status y observar untracked.
- ☐ Ejecutar git add README.md.
- ☐ Repetir git status y explicar el cambio de estado.
- ☐ Crear commit con mensaje apropiado.
- ☐ Modificar README.md.
- ☐ Comparar git diff antes y después de git add.
- ☐ Crear segundo commit.
- ☐ Mostrar git log --oneline.

Producto esperado

Repositorio local con al menos dos commits que representen cambios distintos y cuya historia el estudiante pueda explicar oralmente.

Laboratorio 2 — GitHub y sincronización

Objetivo: conectar un repositorio local con GitHub y practicar push/clone.

- ☐ Crear repositorio remoto vacío.
- ☐ Agregar origen.
- ☐ Ejecutar push -u origin main.
- ☐ Confirmar los commits en GitHub.
- ☐ En otra carpeta, clonar el repositorio.
- ☐ Realizar un cambio desde la copia clonada.
- ☐ Commit y push.
- ☐ Volver a la primera copia y traer el cambio.

Laboratorio 3 — Ramas

Objetivo: desarrollar una funcionalidad aislada.

- ☐ Partir de main actualizado.
- ☐ Crear feature/perfil.
- ☐ Crear al menos dos commits en la rama.
- ☐ Observar el grafo del historial.
- ☐ Integrar a main mediante merge o Pull Request.
- ☐ Eliminar la rama local una vez finalizada.

Laboratorio 4 — Conflicto controlado

Objetivo: provocar y resolver un conflicto deliberadamente para perder el miedo a los merges.

- ☐ Crear dos ramas desde el mismo commit.
- ☐ Modificar la misma línea de README.md de forma distinta.
- ☐ Commit en cada rama.
- ☐ Integrar la primera rama a main.
- ☐ Intentar integrar la segunda.
- ☐ Leer los marcadores de conflicto.
- ☐ Construir una versión final correcta.
- ☐ Agregar, completar el merge y revisar el grafo.

Importante — Este laboratorio debe realizarse en un repositorio de práctica. Provocar conflictos intencionales en el proyecto principal durante una clase puede afectar trabajo real del grupo.

Laboratorio 5 — Flujo completo de equipo

Objetivo: simular una entrega profesional mediante ramas y Pull Requests.

- ☐ Asignar una tarea pequeña a cada integrante.
- ☐ Cada estudiante crea su propia rama.
- ☐ Cada estudiante realiza commits lógicos.
- ☐ Cada rama se publica en GitHub.
- ☐ Cada estudiante crea un Pull Request.
- ☐ Otro integrante revisa y deja al menos una observación útil.
- ☐ El autor corrige si corresponde.
- ☐ Se integra el PR.
- ☐ Todos actualizan main.
- ☐ Se crea un tag de versión de laboratorio.

21. Evaluación práctica sugerida

Una evaluación de Git debería medir comprensión, no memoria mecánica. El estudiante debe demostrar que puede razonar sobre el estado del repositorio.

Criterio	Evidencia esperada	Peso sugerido
Modelo mental	Explica working tree, staging, commit y remoto.	20%
Flujo básico	Realiza add/commit/status/diff correctamente.	20%
Ramas	Crea, publica e integra una rama.	20%
Conflictos	Resuelve un conflicto y justifica la solución.	20%
GitHub	Utiliza PR y explica revisión/merge.	10%
Seguridad	Reconoce secretos y .gitignore.	10%

Preguntas de defensa individual

- ¿Qué diferencia existe entre git add y git commit?
- ¿Qué significa que un archivo esté staged?
- ¿Cuál es la diferencia entre commit y push?
- ¿Qué diferencia existe entre fetch y pull?
- ¿Qué representa origin?
- ¿Por qué no conviene que todos trabajen directamente sobre main?
- ¿Qué ocurre cuando existe un conflicto?
- ¿Qué riesgo tiene git reset --hard?
- ¿Por qué agregar .env a .gitignore no resuelve un secreto que ya fue publicado?
- ¿Cómo demostrarías qué cambios incluye tu rama antes de crear un PR?

22. Hoja de referencia rápida

22.1 Inspección

Comando	Uso
git status	Estado actual del repositorio.
git diff	Cambios no staged.
git diff --staged	Cambios que entrarán al próximo commit.
git log --oneline --graph --decorate	Historia resumida y visual.
git branch --show-current	Nombre de la rama actual.
git remote -v	Remotos configurados.

22.2 Registrar cambios

Comando	Uso
git add archivo	Preparar un archivo.
git add .	Preparar cambios bajo la carpeta actual.
git commit -m "mensaje"	Crear commit.
git commit --amend	Reemplazar último commit local.

22.3 Ramas

Comando	Uso
git branch	Listar ramas.
git switch rama	Cambiar de rama.
git switch -c nueva-rama	Crear y cambiar.
git merge rama	Integrar rama en la actual.
git branch -d rama	Eliminar rama local ya integrada.

22.4 Remotos

Comando	Uso
git clone URL	Clonar repositorio.
git fetch	Descargar referencias remotas.
git pull	Descargar e integrar.
git push	Publicar commits.
git push -u origin rama	Publicar y configurar seguimiento.

22.5 Recuperación

Comando	Uso
git restore archivo	Descartar modificación local del archivo.
git restore --staged archivo	Sacar del staging.
git stash	Guardar cambios temporalmente.
git stash pop	Recuperar último stash.
git revert HASH	Crear commit que revierte otro.

23. Glosario esencial

Término	Definición
Branch / rama	Referencia móvil que permite una línea de trabajo paralela.
Clone	Copia de un repositorio remoto con su historial.
Commit	Registro lógico de cambios en la historia.
Conflict	Situación en la que Git necesita intervención humana para integrar cambios.
Fetch	Descarga referencias y objetos desde el remoto sin integrar automáticamente.
HEAD	Referencia al punto actual de trabajo.
Merge	Integración de historias o ramas.

Término	Definición
Origin	Nombre convencional del remoto principal.
Pull	Obtención e integración de cambios remotos.
Pull Request	Propuesta colaborativa de integración de una rama.
Push	Envío de commits al remoto.
Remote	Referencia configurada hacia otro repositorio.
Repository	Proyecto gestionado por Git junto con su historial.
Staging area	Zona donde se selecciona el contenido del próximo commit.
Tag	Nombre fijo aplicado a un commit para identificar una versión.
Working directory	Archivos actualmente disponibles para edición.

24. Checklist antes de entregar un proyecto

- ☐ git status no muestra cambios accidentales pendientes.
- ☐ El código fuente necesario está versionado.
- ☐ No hay contraseñas, tokens ni secretos en el repositorio.
- ☐ El .gitignore es correcto.
- ☐ El README explica requisitos, instalación y ejecución.
- ☐ Las migraciones o scripts de base de datos están incluidos.
- ☐ El proyecto compila o ejecuta desde una clonación limpia.
- ☐ main contiene la versión estable de entrega.
- ☐ Los Pull Requests importantes fueron revisados.
- ☐ La versión entregada está identificada con un tag o release si así se acordó.
- ☐ Cada integrante puede explicar el historial, ramas y decisiones principales.
- ☐ No existen archivos de conflicto (<<<<<<, =====, >>>>>>) olvidados en el código.

25. Diez reglas para no sufrir con Git

1. Antes de tocar nada: git status.
2. Antes de comenzar una tarea: actualiza main.
3. Una tarea importante: una rama.
4. Un commit: una intención lógica.
5. Antes del commit: revisa diff --staged.
6. Antes del push: entiende qué commits estás enviando.

7. Nunca publiques secretos.
8. No uses --force ni reset --hard si no puedes explicar exactamente el efecto.
9. Un conflicto se resuelve entendiendo el código, no eligiendo botones al azar.
10. **Git es parte del proceso de ingeniería, no una formalidad del final.**

Cierre — El objetivo no es memorizar veinte comandos. El objetivo es desarrollar un modelo mental que permita saber qué está ocurriendo con el código en cada momento. Cuando ese modelo es sólido, Git deja de parecer “peligroso” y se convierte en una red de seguridad para experimentar, colaborar y construir software con disciplina.